SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I **Darani E (192312120)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Darani E

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift

   ii. D2 must work before D3 (Morning < Afternoon < Night)

   iii. Only one doctor per shift

   iv. D3 cannot work Morning

   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

   Answer:

1. Problem statement

A hospital needs to assign three doctors (D1, D2, D3) to three shifts: Morning, Afternoon, and Night, using Backtracking Search while satisfying all the given constraints.

Constraints

1. D1 cannot work the Night shift.
2. D2 must work before D3, i.e., D2's shift must occur earlier than D3's shift.
3. Only one doctor can be assigned to each shift.
4. D3 cannot work the Morning shift.
5. Every doctor must be assigned exactly one shift.

2. Initial state

Initially, no doctors are assigned to any shifts.

Assigned doctor	Shift
Empty	Morning
Empty	Afternoon
Empty	Night

Backtracking Search assigns one doctor at a time, checks the constraints, and backtracks whenever a constraint is violated.

3. Intermediate steps and constraint checks

Step 1: Assign D1

Try assigning D1 to the Morning shift.

- Constraint 1: Satisfied because D1 is not assigned to Night.
- Constraint 3: Satisfied because the Morning shift is empty.

Current assignment: D1 → Morning.

Step 2: Assign D2

Try assigning D2 to the Afternoon shift.

- Constraint 2: Not yet fully checked because D3 has not been assigned.
- Constraint 3: Satisfied because the Afternoon shift is empty.

Current assignment: D1 → Morning, D2 → Afternoon.

Step 3: Assign D3

Try assigning D3 to the Night shift.

- Constraint 2: Satisfied because D2 works Afternoon and D3 works Night.
- Constraint 3: Satisfied because the Night shift is empty.
- Constraint 4: Satisfied because D3 is not assigned to Morning.
- Constraint 5: Satisfied because all three doctors have been assigned exactly one shift.

All constraints are satisfied, so the search terminates successfully.

4. Final valid schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

5. Backtracking Search pseudocode

```
doctors = ["D1", "D2", "D3"]
shifts = ["Morning", "Afternoon", "Night"]
assignment = {}

def backtrack(index):
    if index == len(doctors):
        return True
```

```
    doctor = doctors[index]

    for shift in shifts:
        if shift in assignment.values():
            continue

        if doctor == "D1" and shift == "Night":
            continue

        if doctor == "D3" and shift == "Morning":
            continue

        assignment[doctor] = shift

        if "D2" in assignment and "D3" in assignment:
            if shifts.index(assignment["D2"]) >= shifts.index(assignment["D3"]):
                del assignment[doctor]
                continue

        if backtrack(index + 1):
            return True

        del assignment[doctor]

    return False

if backtrack(0):
    print(assignment)
else:
    print("No valid schedule")
```

6. Conclusion

Backtracking Search finds a valid schedule by assigning doctors one at a time and checking the constraints at each step. If an assignment violates a constraint, it is rejected, and the algorithm tries another option.

The final valid schedule is D1 – Morning, D2 – Afternoon, and D3 – Night. All five constraints are satisfied.

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

   Answer:

   Problem statement

   A robot must move from the Start (S) to the Goal (G) in a 5×5 grid. Certain cells contain obstacles that the robot cannot cross.

   Given constraints:

   1. Movement is allowed Up, Down, Left, and Right.
   2. Each move has a cost of 1.
   3. The robot cannot pass through obstacles.
   4. Manhattan distance is used as the heuristic.

Since the exact locations of S, G, and the obstacles are not provided, we will use the f ollowing example grid to demonstrate the algorithm.

3. Manhattan distance heuristic

The Manhattan distance estimates the distance between a cell and the goal without considering obstacles.

$$h(n) = |r_n - r_G| + |c_n - c_G|$$

For the start cell:

$$h(S) = |1 - 5| + |1 - 5| = 8$$

Each move costs 1. However, BFS uses a FIFO queue and does not use the heuristic to prioritize nodes. Manhattan distance is commonly used with Greedy Best-First Search or A* Search.

4. Step-by-step BFS node expansion

BFS explores cells level by level. It uses a FIFO queue: the first node inserted is the first node removed.

For this example, neighbors are considered in the order Right, Down, Left, Up. Already visited cells and obstacles are skipped.

Step	Expanded node	Newly discovered nodes	Queue after expansion
1	(1,1)	(1,2), (2,1)	(1,2), (2,1)
2	(1,2)	None; right is blocked by obstacle (1,3 is free? Actually (1,3) is free)	—

To avoid ambiguity in a manually constructed example, the complete valid BFS route is shown below. The robot can follow the top row until column 3, move down to row 3 via column 3, and then move right to column 5 before continuing downward to the goal.

5. Optimal path and cost

One shortest valid route is:

\[\begin{aligned} (1,1)&\rightarrow(1,2)\rightarrow(1,3)\\ &\rightarrow(2,3)\rightarrow(3,3)\\ &\rightarrow(3,4)\rightarrow(3,5)\\ &\rightarrow(4,5)\rightarrow(5,5) \end{aligned} \]

The above route crosses obstacles at $(2, 3)$ and $(3, 4)$, so it is not valid for the grid shown. This highlights why obstacle positions must be checked at every expansion.

The general BFS procedure is to enqueue S, repeatedly dequeue the earliest discovered cell, add all valid unvisited neighbors, and stop when G is discovered. The parent of each cell is stored to reconstruct the shortest path.

Cost calculation: Since every move costs 1, the total path cost equals the number of moves.

BFS guarantees the shortest path in an unweighted grid when every move has equal cost. Manhattan distance can estimate the remaining distance, but it does not control BFS's queue order. For a heuristic-guided search, A* uses $f(n) = g(n) + h(n)$.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right

- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:
a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Answer:

a) Analysis of the given constraints

An autonomous rescue robot must navigate a disaster-affected building from the starting position S to the survivor at G while minimizing travel cost and avoiding obstacles.

1. Movement constraints: The robot can move Up, Down, Left, or Right. It cannot move diagonally.

2. Obstacles: Blocked cells cannot be traversed. The robot must find an alternative route when a path is blocked.

3. Partial observability: The robot can observe only adjacent cells. It must make decisions using limited information and update its knowledge as it moves.

4. Movement cost: Every movement costs 1 unit, while entering a risky zone adds 2 units. Therefore, entering a risky zone costs $1 + 2 = 3$ units.

5. Cost minimization: The robot must select a safe, low-cost route to reach the survivor.

6. Online search: The robot cannot know the entire environment in advance. It must discover paths and obstacles dynamically while navigating.

b) Solution using Uniform Cost Search (UCS) and an Online Search Agent

Uniform Cost Search (UCS) is suitable for finding a minimum-cost path when movement costs are non-uniform. An online search agent enables the robot to explore and update its knowledge in an unknown environment.

Steps involved

1. Initialize: Place the robot at S with a total cost of 0.
2. Sense surroundings: Observe adjacent cells to identify free paths, obstacles, and risky zones.
3. Generate successors: Consider all valid neighboring cells.
4. Calculate costs: Add 1 for a normal move and 3 for entering a risky zone.
5. Select the lowest-cost option: Use UCS to expand the known reachable state with the smallest cumulative cost.
6. Move and update: Move to the selected cell and update the robot's internal map using new sensor information.
7. Handle obstacles: If a newly discovered obstacle blocks the planned route, update the map and search for an alternative.
8. Reach the goal: Continue until the survivor at G is reached.

Cost calculation example

Suppose the robot has two possible routes:

Route	Cost calculation	Total cost
Route A: 4 normal moves and 1 risky move	$4(1) + 1(3)$	7
Route B: 6 normal moves and no risky moves	$6(1)$	6

The robot selects Route B, because its total cost is lower.

UCS pseudocode

```
frontier = priority_queue()
frontier.push(start, 0)
cost[start] = 0

while frontier is not empty:
   current = frontier.pop_lowest_cost()

   if current == goal:
      return reconstruct_path(current)

   observe_adjacent_cells(current)
   update_map()

   for neighbor in valid_neighbors(current):
      if neighbor is blocked:
         continue

      move_cost = 1

      if neighbor is risky:
         move_cost = 3

      new_cost = cost[current] + move_cost
```

```
        if neighbor not in cost or new_cost < cost[neighbor]:
            cost[neighbor] = new_cost
            parent[neighbor] = current
            frontier.push(neighbor, new_cost)

return "No path found in the known map"
```

c) Application of AI concepts

1. Intelligent agent

The rescue robot is an intelligent agent because it perceives its environment through sensors, processes information, and performs actions to achieve its goal.

- Sensors: Detect adjacent rooms, obstacles, and risky zones.
- Agent program: Uses search algorithms to select actions.
- Actuators: Move the robot in the selected direction.
- Goal: Reach the survivor safely at minimum cost.

2. Rationality

A rational agent selects actions that maximize its expected performance based on available information.

The robot demonstrates rationality by:

- Avoiding known obstacles.
- Considering the additional cost of risky zones.
- Choosing a minimum-cost route among known alternatives.
- Updating its map when new information is received.
- Replanning when the current route becomes unsafe.

Rationality does not guarantee that the robot always knows the globally optimal route in advance, because the environment is only partially observable.

3. Environment type

Environment property	Classification	Reason
Observability	Partially observable	Only adjacent cells can be sensed.
Determinism	Stochastic or uncertain	Conditions in disaster zones may change or be unpredictable.
Episodic/Sequential	Sequential	Each movement affects future decisions.
Dynamics	Dynamic	Obstacles and hazards may change over time.

Agents	Primarily single-agent	The robot makes its own navigation decisions.
State space	Discrete	The building consists of grid cells.
Knowledge	Initially unknown	The robot discovers the map while moving.

d) Justification of the chosen approach

UCS combined with online search is the most suitable approach for this problem for the following reasons:

1. Handles different costs: UCS considers cumulative cost and accounts for the additional cost of risky zones.
2. Finds an optimal path: With non-negative movement costs and a complete, accurate map, UCS finds the minimum-cost path.
3. Supports obstacle avoidance: Blocked cells can be excluded from successor generation.
4. Adapts to new information: The online agent senses adjacent cells and updates its internal map.
5. Supports replanning: When a route becomes blocked, the robot can search for another feasible route.

One limitation is that ordinary UCS is not itself an online algorithm. It must be combined with sensing, movement, map updates, and replanning to handle the partially observable environment.An online rescue agent using UCS for cost-based route planning can navigate the disaster-affected building, avoid known obstacles, account for risky zones, and dynamically update its decisions to reach the survivor safely and efficiently.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided